AI для .NET-розробника

Том 4 · Агенти й MCP

Розділи 10–12

Розділ 10. MCP — Model Context Protocol

10.1. Проблема й рішення

Без стандарту кожен AI-клієнт мав би власну інтеграцію з GitHub, базами, браузером тощо. **MCP** — відкритий протокол, за яким **MCP-сервер** «виставляє» можливості, а **MCP-клієнт** (Copilot, Claude, Cursor…) їх використовує. Один сервер — багато клієнтів.

10.2. Що надає MCP-сервер

Примітив	Суть	Приклад
Tools	Функції, які модель може викликати	`create_issue`, `run_sql_query`
Resources	Дані для читання як контекст	Файл, схема БД, документ
Prompts	Шаблони запитів сервера	«Проаналізуй лог»

Найчастіше ви використовуватимете **tools**.

10.3. Транспорти

Тип	Як працює	Типово для
stdio	Клієнт запускає локальний процес і говорить через стандартні потоки	Локальні інструменти (`npx`, `dotnet`, `docker`)
HTTP (streamable HTTP; раніше SSE)	Віддалений сервер за URL, часто з OAuth	Хмарні сервіси (GitHub, Azure тощо)

10.4. Підключення

Конфігурація — JSON. У VS Code — `.vscode/mcp.json` (на рівні проєкту) або налаштування користувача; Visual Studio читає файл конфігурації MCP у рішенні/користувача; Copilot CLI — власний конфіг у домашній папці; Claude Code — `.mcp.json`. Точні шляхи й ключі звіряйте з документацією версії, яку використовуєте.

```
// приклад у стилі VS Code (.vscode/mcp.json)
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest"]
    }
  }
}
```

> **Ніколи** не клали токени прямо в JSON, що йде в git. Використовуйте prompts/inputs клієнта, змінні середовища або OAuth.

10.5. Корисні MCP-сервери для .NET-розробника

Сервер	Навіщо
GitHub MCP	Issue, PR, пошук коду, Actions з чату
Microsoft Learn / Docs MCP	Актуальна документація .NET та Azure — зменшує вигадки про API
Playwright MCP	Керування браузером: перевірка UI вашого веб-застосунку
Бази даних (SQL Server, PostgreSQL)	Дивитися схему, виконувати запити (краще read-only!)
Azure MCP	Керування ресурсами Azure
Власний сервер на C#	Інтеграція з вашою внутрішньою системою

10.6. Свій MCP-сервер на C#

Існує офіційний C# SDK (`ModelContextProtocol` у NuGet). Скелет — консольний застосунок із stdio-транспортом і атрибутами для інструментів:

```
// dotnet new console -n MyMcp ; dotnet add package ModelContextProtocol --prerelease ;
dotnet add package Microsoft.Extensions.Hosting
var builder = Host.CreateApplicationBuilder(args);
builder.Services
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();
await builder.Build().RunAsync();

[McpServerToolType]
public static class TimeTools
{
    [McpServerTool, Description("Повертає поточний UTC-час у форматі ISO 8601")]
    public static string UtcNow() => DateTime.UtcNow.ToString("O");
}
```

API SDK активно змінюється — під час практики перевіряйте офіційний README та приклади.

10.7. Безпека MCP

- MCP-сервер виконує код або має доступ до даних — ставте лише довірені, дивіться першоджерело.
- Принцип мінімальних прав: токени з обмеженим scope, БД лише для читання.
- Вибірково вмикайте інструменти; менше інструментів — менше плутанини й ризику.
- **Prompt injection через дані:** вміст з issue чи веб-сторінки може містити приховані команди для агента.
- Багато інструментів з'їдають контекст: описи всіх tools додаються до кожного запиту.

> **Завдання 10.** Підключіть GitHub MCP (або Microsoft Learn MCP) у вашому IDE. Попросіть: «Покажи останні 3 коміти репозиторію AI-Copilot-Learning-Guide» або «Згідно з документацією, як налаштувати Rate Limiting в ASP.NET Core?».

Розділ 11. Агентна робота та Copilot CLI

11.1. Цикл агента

```
Задача → [ Думка → Виклик інструмента → Результат ] × N → Відповідь
Інструменти: читання/пошук файлів, редагування, термінал, MCP, веб-пошук
```

Агент сам коригує дії за результатами: збірка впала → читає помилку → правка → ще раз збирає. Тому критично, щоб у проєкті була **перевірка**, за якою агент розуміє «готово»: збірка, тести, лінтер.

11.2. Як ставити задачі агенту

Елемент	Приклад
Ціль	«Додати фільтрацію /todos?isDone=true»
Критерії готовності	«dotnet build без попереджень, нові тести проходять, Swagger оновлено»
Межі	«Не чіпати Migrations та AuthController»
Підказки	«Дивись, як зроблено фільтр у /projects»

Робота у стилі **spec-driven**: спершу короткий опис вимог у Markdown (`docs/spec-xxx.md`), агент уточнює питання, складає план, і лише потім реалізує. Це різко підвищує якість для великих фіч.

11.3. Copilot CLI

GitHub Copilot CLI — агент у терміналі. Працює в поточній директорії, читає код, виконує команди, редагує файли, підтримує інструкції (`AGENTS.md`, `.github/copilot-instructions.md`), MCP, skills і custom agents. Встановлення й команди — за актуальною документацією GitHub (зазвичай через npm, WinGet чи інший менеджер пакетів; потрібна активна підписка Copilot).

```
cd C:\src\TodoApi
copilot            # запустити інтерактивну сесію
> Знайди всі місця без CancellationToken і запропонуй план виправлення
> /help
```

Корисне	Опис
Slash-команди	Керування сесією, моделлю, контекстом, MCP (список — `/help`)
Режим плану	Спочатку обговорення плану, потім виконання
Дозволи	Запитує дозвіл на команди й шляхи; можна обмежити allow/deny-списками
Скрипти	Неінтерактивний запуск для автоматизації (див. `copilot --help`)

11.4. Паралельна й багатоагентна робота

- **Субагенти:** дослідження/рев'ю в окремому контексті.
- **Git worktrees:** кілька агентів працюють над різними задачами в окремих копіях репозиторію без конфліктів.
- **Правило:** паралельте *незалежні* задачі; залежні — по черзі.

11.5. Контроль якості роботи агента

1. Читайте diff, не тільки підсумок агента (він може бути надто оптимістичним).
2. Запускайте тести самостійно.
3. Просіть самоперевірку: «Що ти не перевірив? Які ризики?».
4. Для критичного коду — друга пара очей (людина або агент-рев'юер).

> **Завдання 11.** Встановіть Copilot CLI. У проєкті TodoApi дайте задачу з чіткими критеріями готовності, дозвольте лише `dotnet build` / `dotnet test`. Оцініть результат за списком 11.5.

Розділ 12. Cloud agent: від issue до Pull Request

12.1. Що це

Copilot cloud agent (раніше «coding agent») працює у хмарі GitHub: ви призначаєте йому issue або ставите задачу в чаті, він створює гілку в ізольованому середовищі, вносить зміни, запускає збірку/тести й відкриває **Pull Request** для вашого рев'ю. Ви не чекаєте — це фонова робота.

12.2. Коли ефективний

Підходить	Не підходить
Невеликі чітко описані баги й фічі	Розмиті вимоги
Додати тести, оновити документацію	Архітектурні рішення з нуля
Рутинні міграції, чистка техборгу	Задачі, потрібні секрети/прод-доступ

12.3. Як підготувати репозиторій

- **copilot-instructions.md / AGENTS.md** з командами збірки й тестів.
- **Файл налаштування середовища** (`.github/workflows/copilot-setup-steps.yml`): що встановити перед стартом агента — .NET SDK, залежності.
- **CI на PR** (GitHub Actions) — агент і ви бачите результат перевірок.
- **Захист гілок**: обов'язкове рев'ю, щоб агент не міг злити сам.

```
# .github/workflows/copilot-setup-steps.yml (спрощено)
name: "Copilot Setup Steps"
on: workflow_dispatch
jobs:
  copilot-setup-steps:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '9.0.x'
      - run: dotnet restore
```

12.4. Робочий цикл

1. Створіть issue: опис, критерії приймання, посилання на файли.
2. Призначте Copilot (Assignee) або делегуйте з чату/IDE.
3. Агент відкриває draft PR і пише логи роботи.
4. Ви рев'юїте, залишаєте коментарі — агент відповідає правками.
5. Перевіряєте CI, зливаєте самі.

12.5. Гарні issue для агента

```
Заголовок: Додати фільтр isDone в GET /todos
Опис: параметр ?isDone=true|false; без параметра — усі.
Приймання: - тести в tests/TodoApi.Tests/TodoEndpointsTests.cs
           - оновлено README (розділ API)
           - dotnet build/test без помилок
Не чіпати: Migrations/
```

12.6. Безпека й контроль

- Агент працює під обмеженими правами й у пісочниці з обмеженим мережевим доступом (налаштовується).
- Його PR не мають самозатверджуватися; завжди людина-рев'юер.
- Слідкуйте за витратою квот; задачі краще робити невеликими.
- Не пишіть у issue секрети — їх читатиме агент і історія.

> **Завдання 12.** У репозиторії зі своїм TodoApi створіть issue за зразком 12.5 і делегуйте cloud agent (якщо доступно за вашим тарифом). Перегляньте PR, залиште два коментарі.

> **Далі — Том 5:** практика .NET: тести, рефакторинг, безпека та власні AI-застосунки.